

Chapter 13 — Stack

0. What You Will Learn in This Chapter

A **stack** is one of the most common data structures in coding interviews.

It follows the rule:

Last In, First Out

This means the most recently added item is the first one removed.

Stacks are useful when a problem involves:

- matching parentheses
- undoing recent actions
- processing nested structures
- evaluating expressions
- reversing order
- simulating call stacks
- simplifying paths
- removing adjacent duplicates
- handling collisions
- maintaining temporary unresolved state

By the end of this chapter, you should understand:

- What a stack is
- Why stack exists
- When to use stack
- When not to use stack
- How C++ stack works
- How to use vector as a stack
- How to solve parentheses problems
- How to solve expression evaluation problems
- How to solve nested string decoding
- How to implement Min Stack
- How to simulate stack behavior
- How to identify stack problems in interviews
- Common stack mistakes and edge cases

Core problems covered in this chapter:

1. Valid Parentheses
 2. Min Stack
 3. Baseball Game
 4. Backspace String Compare
 5. Remove All Adjacent Duplicates in String
 6. Simplify Path
 7. Evaluate Reverse Polish Notation
 8. Decode String
 9. Asteroid Collision
 10. Implement Queue using Stacks
 11. Implement Stack using Queues
 12. Basic Calculator II
-

1. What Is a Stack?

A stack is a data structure where elements are added and removed from the same end.

The main operations are:

push(x)	add item to top
pop()	remove top item
top()	view top item
empty()	check if stack has no items
size()	number of items

Example:

```
stack<int> st;

st.push(10);
st.push(20);
st.push(30);

cout << st.top(); // 30

st.pop();

cout << st.top(); // 20
```

The last item inserted was 30, and it came out first.

That is why stack is called:

Last In, First Out

2. Why Stack Exists

Stack exists because many problems require processing the **most recent unresolved item first**.

Example:

Valid Parentheses:

```
s = "{[()]}"
```

When we see an opening bracket, we do not know yet when it will close.

So we store it.

When we see a closing bracket, it must match the **most recent** opening bracket.

That is exactly stack behavior.

Example:

```
open: (
open: {
```

```
open: [  
close: ] must match [  
close: } must match {  
close: ) must match (
```

The most recent open bracket is resolved first.
That is stack.

3. Core Need / Why Use Stack Over Other Patterns?

Use stack when the latest thing you saw is the first thing you need to resolve.

3.1 Why Not Basic Traversal?

Basic traversal can scan characters, but it cannot remember nested unresolved states naturally.

For:

```
"({[]})"
```

You need to remember the order of opening brackets.

A few variables are not enough.

Use stack.

3.2 Why Not Queue?

Queue is First In, First Out.

But parentheses need Last In, First Out.

Example:

```
([])
```

The first open bracket is (, but the first closing bracket is].

You need to compare] with the most recent opening bracket [, not the oldest one.

So queue is wrong.

3.3 Why Not Hash Map Alone?

Hash map can store matching pairs:

```
'-' -> '('  
'-' -> '['  
'-' -> '{'
```

But it cannot track nesting order by itself.

Stack tracks the order.

Hash map may support stack, but does not replace it.

3.4 Why Not Recursion?

Recursion uses the call stack internally.

But for many iterative problems, an explicit stack is clearer and safer.

Examples:

- Valid Parentheses
- Simplify Path
- Decode String

- Asteroid Collision
-

4. When to Use Stack

Use stack when the problem involves one of these clues.

4.1 Matching Pairs

Problem has:

- parentheses
- brackets
- opening and closing symbols
- XML/HTML tags
- nested delimiters

Examples:

- Valid Parentheses
- Minimum Remove to Make Valid Parentheses
- Score of Parentheses

4.2 Undo / Backtracking Recent Action

Problem has:

- backspace
- undo
- remove previous character
- cancel previous item

Examples:

- Backspace String Compare
- Baseball Game
- Remove All Adjacent Duplicates

4.3 Nested Structures

Problem has:

- nested strings
- nested lists
- recursive expressions
- decode format

Examples:

- Decode String
- Mini Parser
- Number of Atoms

4.4 Expression Evaluation

Problem has:

- operators
- operands
- postfix notation
- calculator

Examples:

- Evaluate Reverse Polish Notation
- Basic Calculator
- Basic Calculator II

4.5 Path / Token Processing

Problem has path-like strings:

`"/a/./b/../../c/"`

Use stack to keep valid directory names.

Example:

- Simplify Path

4.6 Collision / Cancellation

Problem involves items moving and canceling each other.

Example:

- Asteroid Collision

Stack stores surviving previous objects.

5. When NOT to Use Stack

5.1 Need First-In, First-Out Processing

Use queue.

Examples:

- BFS
- level order traversal
- shortest path in unweighted graph

5.2 Need Current Minimum / Maximum Over Sliding Window

Use monotonic deque, not normal stack.

Example:

- Sliding Window Maximum

5.3 Need Sorted Order

Use heap, set, map, or sorting.

Stack does not maintain sorted order.

5.4 Need Fast Lookup

Use hash map or hash set.

Stack only gives access to the top.

5.5 Need Next Greater / Next Smaller

That is usually **Monotonic Stack**, which is a specialized stack pattern.

It comes in the next chapter.

6. Core Intuition

A stack stores unfinished work.

At each element, ask:

13. Does this start something?
14. Does this end something?
15. Does this cancel the previous thing?
16. Does this depend on the most recent unresolved item?

If yes, stack is likely useful.

Common stack pattern:

for each item:

if item starts something:

push it

else if item resolves something:
compare with stack top
pop if valid

7. Mental Models

7.1 Stack of Plates

You place plates one on top of another.
The last plate added is the first one removed.

7.2 Unfinished Work

Opening bracket means:

I am waiting for a matching closing bracket.

Push it.

Closing bracket means:

Resolve the most recent unfinished opening bracket.

Pop it.

7.3 Undo History

Backspace removes the most recent character.
So a stack naturally models typed characters.

7.4 Nested Box Model

For nested strings like:

3[a2[c]]

You enter a new nested box at [, and exit it at].
Stack remembers previous state.

8. Stack vs Queue vs Heap

Stack	Last In, First Out	parentheses, undo, nested parsing
Queue	First In, First Out	BFS, level order, scheduling
Heap	priority-based	repeatedly get min/max
Deque	both ends	sliding window maximum, monotonic queue

9. C++ Stack Tools Needed

9.1 Standard Stack

```
stack<int> st;
```

Operations:

```
st.push(x);
st.pop();
st.top();
st.empty();
st.size();
```

Important:

pop() does not return the removed value.

Wrong:

```
int x = st.pop(); // invalid
```

Correct:

```
int x = st.top();
st.pop();
```

9.2 Vector as Stack

Sometimes vector is more flexible:

```
vector<int> st;

st.push_back(x);
st.back();
st.pop_back();
st.empty();
```

Use vector when you need iteration or easier result building.

9.3 String as Stack

For character problems:

```
string st;

st.push_back(c);
st.back();
st.pop_back();
```

Very useful for:

- Remove Adjacent Duplicates
- Backspace String Compare
- building processed strings

10. Core Stack Templates

Template 1 — Basic Stack Scan

```
stack<int> st;

for (int x : nums) {
```

```
    if (/* should push */) {  
        st.push(x);  
    } else {  
        if (!st.empty()) {  
            st.pop();  
        }  
    }  
}
```

Template 2 — Parentheses Matching

```
bool isValid(string s) {  
    stack<char> st;  
  
    for (char c : s) {  
        if (c == '(' || c == '[' || c == '{') {  
            st.push(c);  
        } else {  
            if (st.empty()) {  
                return false;  
            }  
  
            char top = st.top();  
            st.pop();  
  
            if ((c == ')' && top != '(') ||  
                (c == ']' && top != '[') ||  
                (c == '}' && top != '{')) {  
                return false;  
            }  
        }  
    }  
  
    return st.empty();  
}
```

Template 3 — String as Stack

```
string st;  
  
for (char c : s) {  
    if (!st.empty() && st.back() == c) {  
        st.pop_back();  
    }  
}
```

```
    } else {  
        st.push_back(c);  
    }  
}  
  
return st;
```

Template 4 — Evaluate Postfix Expression

```
stack<int> st;  
  
for (string token : tokens) {  
    if (token is number) {  
        st.push(number);  
    } else {  
        int b = st.top(); st.pop();  
        int a = st.top(); st.pop();  
  
        int result = apply(a, b, token);  
        st.push(result);  
    }  
}  
  
return st.top();
```

Important:

For subtraction and division, order matters:

a - b
a / b

not:

b - a
b / a

Template 5 — Stack for Path Simplification

```
vector<string> st;  
  
for each token in path split by '/':  
    if token is empty or ".":  
        ignore  
    else if token is "..":  
        if stack not empty:  
            pop
```

```
else:  
    push token
```

Template 6 — Nested Decode Stack

```
stack<int> countStack;  
stack<string> stringStack;  
  
string current = "";  
int number = 0;  
  
for (char c : s) {  
    if (isdigit(c)) {  
        number = number * 10 + (c - '0');  
    } else if (c == '[') {  
        countStack.push(number);  
        stringStack.push(current);  
        number = 0;  
        current = "";  
    } else if (c == ']') {  
        int repeat = countStack.top();  
        countStack.pop();  
  
        string previous = stringStack.top();  
        stringStack.pop();  
  
        while (repeat-- > 0) {  
            previous += current;  
        }  
  
        current = previous;  
    } else {  
        current += c;  
    }  
}
```

11. Major Variations of Stack Problems

Matching symbols	push open, pop on close	Valid Parentheses
Undo simulation	remove most recent valid item	Backspace String
Score/history	store previous scores	Baseball Game
Path stack	push directories, pop on ..	Simplify Path
Expression stack	operands/operators	Reverse Polish Notation
Nested parsing	stack previous state	Decode String
Collision stack	compare with previous survivor	Asteroid Collision
Min tracking	stack with current minimum	Min Stack
Two stacks	simulate queue	Implement Queue

12. Full Problem Walkthrough 1 — LeetCode 20: Valid Parentheses

12.1 Problem Summary

Given a string containing:

```
'(', ')', '[', ']', '{', '}'
```

Return true if brackets are valid.

Valid means:

17. Every opening bracket has a matching closing bracket.
18. Brackets close in correct order.

Example:

```
s = "()[]{}"
```

Output:

```
true
```

Example:

```
s = "["
```

Output:

```
false
```

12.2 How to Identify This Pattern

The problem has:

- opening brackets
- closing brackets
- nested matching
- most recent opening bracket must close first

This is a classic stack problem.

12.3 Brute Force Idea

Repeatedly remove valid pairs:

```
()  
[]  
{}
```

until no more can be removed.

If string becomes empty, valid.

But this can be inefficient and awkward.

12.4 Optimized Intuition

Use stack.

- Push opening brackets.
 - When seeing closing bracket, compare with stack top.
 - If mismatch, return false.
 - At end, stack must be empty.
-

12.5 C++ Code

```
class Solution {  
public:  
    bool isValid(string s) {  
        stack<char> st;  
  
        for (char c : s) {  
            if (c == '(' || c == '[' || c == '{') {  
                st.push(c);  
            } else {  
                if (st.empty()) {  
                    return false;  
                }  
  
                char top = st.top();  
                st.pop();  
  
                if (c == ')' && top != '(') {  
                    return false;  
                }  
            }  
        }  
    }  
};
```

```

        if (c == ']' && top != '[') {
            return false;
        }

        if (c == '}' && top != '{') {
            return false;
        }
    }

    return st.empty();
}
};

```

12.6 Dry Run

Input:

s = "{[()]}"

(	push	(
{	push	(, {
[	push	(, {, [
]	matches [	(, {
}	matches {	(
)	matches (	empty

Return true.

12.7 Complexity

Time:

O(n)

Each character is processed once.

Space:

O(n)

In worst case, all characters are opening brackets.

12.8 Edge Cases

- Empty string
 - Only opening brackets
 - Only closing brackets
 - Wrong order: ([])]
 - Nested valid brackets
 - Multiple valid groups
-

12.9 Common Mistakes

Mistake 1 — Not checking empty stack before top()

Wrong:

```
char top = st.top();
```

If stack is empty, this crashes.

Correct:

```
if (st.empty()) {  
    return false;  
}
```

Mistake 2 — Returning true without checking leftover openings

Input:

```
"(((("
```

Stack is not empty.

Return false.

12.10 Similar Problems

- Minimum Remove to Make Valid Parentheses
 - Score of Parentheses
 - Generate Parentheses
 - Remove Invalid Parentheses
-

13. Full Problem Walkthrough 2 — LeetCode 155: Min Stack

13.1 Problem Summary

Design a stack that supports:

```
push(val)  
pop()  
top()  
getMin()
```

All operations must be $O(1)$.

13.2 How to Identify This Pattern

Normal stack gives $O(1)$ top.

But getMin() needs current minimum.
We need to remember minimum at each stack state.
Use an auxiliary stack.

13.3 Intuition

Maintain two stacks:

19. st stores all values.
20. minSt stores minimum so far at each level.

When pushing value x, push:

`min(x, currentMin)`

onto minSt.

13.4 C++ Code

```
class MinStack {
private:
    stack<int> st;
    stack<int> minSt;

public:
    MinStack() {
    }

    void push(int val) {
        st.push(val);

        if (minSt.empty()) {
            minSt.push(val);
        } else {
            minSt.push(min(val, minSt.top()));
        }
    }

    void pop() {
        st.pop();
        minSt.pop();
    }

    int top() {
        return st.top();
    }

    int getMin() {
```

```

        return minSt.top();
    }
};

```

13.5 Dry Run

Operations:

```

push(-2)
push(0)
push(-3)
getMin()
pop()
top()
getMin()

```

push -2	-2	-2	
push 0	-2, 0	-2, -2	
push -3	-2, 0, -3	-2, -2, -3	
getMin			-3
pop	-2, 0	-2, -2	
top			0
getMin			-2

13.6 Complexity

Each operation:

$O(1)$

Space:

$O(n)$

13.7 Common Mistakes

- Only storing new minimum values and mishandling duplicates
- Forgetting to pop from minSt
- Trying to scan stack for minimum, which is $O(n)$

13.8 Similar Problems

- Max Stack
- Design Browser History
- Stock Spanner, monotonic stack later

14. Full Problem Walkthrough 3 — LeetCode 682: Baseball Game

14.1 Problem Summary

You are given operations representing scores.

Rules:

integer	record score
+	record sum of previous two scores
D	record double previous score
C	remove previous score

Return total score.

14.2 How to Identify Stack

The problem repeatedly refers to the previous score or previous two scores.

Also C cancels the most recent score.

This is stack behavior.

14.3 C++ Code

```
class Solution {
public:
    int calPoints(vector<string>& operations) {
        vector<int> st;

        for (string op : operations) {
            if (op == "C") {
                st.pop_back();
            } else if (op == "D") {
                st.push_back(2 * st.back());
            } else if (op == "+") {
                int n = st.size();
                st.push_back(st[n - 1] + st[n - 2]);
            } else {
                st.push_back(stoi(op));
            }
        }
    }
}
```

```

    int total = 0;

    for (int score : st) {
        total += score;
    }

    return total;
}
};

```

14.4 Dry Run

Input:

["5","2","C","D","+"]

5	push 5	5
2	push 2	5, 2
C	remove 2	5
D	push 10	5, 10
+	push 15	5, 10, 15

Total:

30

14.5 Complexity

Time:

$O(n)$

Space:

$O(n)$

15. Full Problem Walkthrough 4 — LeetCode 844: Backspace String Compare

15.1 Problem Summary

Given two strings where # means backspace, return whether they are equal after processing.

Example:

```
s = "ab#c"
t = "ad#c"
```

Both become:

```
"ac"
```

Return true.

15.2 How to Identify Stack

Typing characters and pressing backspace is an undo operation.

Backspace removes the most recent valid character.

Use stack.

15.3 C++ Code

```
class Solution {
public:
    string build(string s) {
        string st;

        for (char c : s) {
            if (c == '#') {
                if (!st.empty()) {
                    st.pop_back();
                }
            } else {
                st.push_back(c);
            }
        }

        return st;
    }

    bool backspaceCompare(string s, string t) {
        return build(s) == build(t);
    }
};
```

15.4 Complexity

Let n and m be string lengths.

Time:

$O(n + m)$

Space:

$O(n + m)$

There is also an $O(1)$ two-pointer solution, but stack is simpler.

15.5 Edge Cases

- Extra backspaces
- Both strings become empty
- One string becomes empty
- Consecutive #

16. Full Problem Walkthrough 5 — LeetCode 1047: Remove All Adjacent Duplicates in String

16.1 Problem Summary

Given a string, repeatedly remove adjacent equal characters.

Example:

`s = "abbaca"`

Process:

`abbaca -> aaca -> ca`

Output:

`"ca"`

16.2 How to Identify Stack

When current character equals previous character, both cancel.

The previous character is the stack top.

16.3 C++ Code

```
class Solution {
public:
    string removeDuplicates(string s) {
        string st;

        for (char c : s) {
            if (!st.empty() && st.back() == c) {
                st.pop_back();
            } else {
```

```

        st.push_back(c);
    }
}

return st;
}
};

```

16.4 Dry Run

s = "abbaca"

a	push	a
b	push	ab
b	pop b	a
a	pop a	empty
c	push	c
a	push	ca

Return:

"ca"

16.5 Complexity

Time:

$O(n)$

Space:

$O(n)$

16.6 Similar Problems

- Remove All Adjacent Duplicates in String II
- Make The String Great
- Backspace String Compare

17. Full Problem Walkthrough 6 — LeetCode 71: Simplify Path

17.1 Problem Summary

Given a Unix-style path, simplify it.

Rules:

.	current directory
..	parent directory
empty	ignore repeated slashes
normal name	directory

Example:

path = "/a/./b/../../c/"

Output:

"/c"

17.2 How to Identify Stack

Path navigation uses recent directories.

When seeing .., go back to the most recent directory.

Use stack.

17.3 C++ Code

```
class Solution {
public:
    string simplifyPath(string path) {
        vector<string> st;
        string token;
        stringstream ss(path);

        while (getline(ss, token, '/')) {
            if (token == "" || token == ".") {
                continue;
            } else if (token == "..") {
                if (!st.empty()) {
                    st.pop_back();
                }
            } else {
                st.push_back(token);
            }
        }

        string result = "";
```

```

    for (string dir : st) {
        result += "/";
        result += dir;
    }

    return result.empty() ? "/" : result;
}
};

```

17.4 Dry Run

Input:

"/a/./b/./../c/"

Tokens:

a, ., b, .., .., c

a	push	a
.	ignore	a
b	push	a, b
..	pop b	a
..	pop a	empty
c	push	c

Return:

"/c"

17.5 Complexity

Time:

$O(n)$

Space:

$O(n)$

17.6 Common Mistakes

- Treating ... as ..
- Forgetting repeated slashes
- Returning empty string instead of /
- Popping when stack is empty

18. Full Problem Walkthrough 7 — LeetCode 150: Evaluate Reverse Polish Notation

18.1 Problem Summary

Evaluate an expression in Reverse Polish Notation.

Example:

```
tokens = ["2","1","+","3","*"]
```

This means:

```
(2 + 1) * 3 = 9
```

Output:

```
9
```

18.2 How to Identify Stack

In postfix notation:

- numbers are pushed
- operators apply to most recent two numbers

This is stack behavior.

18.3 C++ Code

```
class Solution {
public:
    int evalRPN(vector<string>& tokens) {
        stack<int> st;

        for (string token : tokens) {
            if (token == "+" || token == "-" || token == "*" || token == "/") {
                int b = st.top();
                st.pop();

                int a = st.top();
                st.pop();

                if (token == "+") {
                    st.push(a + b);
                } else if (token == "-") {
                    st.push(a - b);
                } else if (token == "*") {
```

```

        st.push(a * b);
    } else {
        st.push(a / b);
    }
    } else {
        st.push(stoi(token));
    }
}

return st.top();
}
};

```

18.4 Dry Run

Input:

["2","1","+","3","*"]

2	push	2
1	push	2, 1
+	2 + 1	3
3	push	3, 3
*	3 * 3	9

Return 9.

18.5 Important Mistake

For subtraction:

a - b

not:

b - a

For division:

a / b

not:

b / a

19. Full Problem Walkthrough 8 — LeetCode 394: Decode String

19.1 Problem Summary

Decode strings like:

"3[a2[c]]"

Output:

"accaccacc"

Rules:

k[encoded_string]

means repeat encoded string k times.

19.2 How to Identify Stack

The problem has:

- nested brackets
- repeated substrings
- need to remember previous string before entering bracket

Use stacks.

19.3 C++ Code

```
class Solution {
public:
    string decodeString(string s) {
        stack<int> countStack;
        stack<string> stringStack;

        string current = "";
        int number = 0;

        for (char c : s) {
            if (isdigit(c)) {
                number = number * 10 + (c - '0');
            } else if (c == '[') {
                countStack.push(number);
                stringStack.push(current);

                number = 0;
                current = "";
            } else if (c == ']') {
                int repeat = countStack.top();
```

```

        countStack.pop();

        string previous = stringStack.top();
        stringStack.pop();

        string expanded = "";

        for (int i = 0; i < repeat; i++) {
            expanded += current;
        }

        current = previous + expanded;
    } else {
        current += c;
    }
}

return current;
}
};

```

19.4 Dry Run Summary

Input:

"3[a2[c]]"

Process:

2[c] -> cc
a2[c] -> acc
3[acc] -> accaccacc

Return:

"accaccacc"

19.5 Complexity

Let output length be N.

Time:

$O(N)$

Space:

$O(N)$

19.6 Common Mistakes

- Not handling multi-digit numbers
 - Not resetting number after [
 - Losing previous string
 - Failing nested brackets
-

20. Full Problem Walkthrough 9 — LeetCode 735: Asteroid Collision

20.1 Problem Summary

Asteroids move along a line.

- Positive means moving right.
- Negative means moving left.

When a right-moving asteroid meets a left-moving asteroid, they collide.

Smaller one explodes.

If equal, both explode.

Example:

```
asteroids = [5,10,-5]
```

Output:

```
[5,10]
```

20.2 How to Identify Stack

A new left-moving asteroid can collide with previous right-moving asteroids.

The most recent surviving right-moving asteroid is checked first.

Use stack.

20.3 C++ Code

```
class Solution {
public:
    vector<int> asteroidCollision(vector<int>& asteroids) {
        vector<int> st;

        for (int asteroid : asteroids) {
            bool alive = true;

            while (alive &&
                  asteroid < 0 &&
                  !st.empty() &&
                  st.back() > 0) {

                if (st.back() < -asteroid) {
                    st.pop_back();
                } else if (st.back() == -asteroid) {
```

```

        st.pop_back();
        alive = false;
    } else {
        alive = false;
    }
}

if (alive) {
    st.push_back(asteroid);
}

return st;
}
};

```

20.4 Dry Run

Input:

[5,10,-5]

5	push	5
10	push	5, 10
-5	collides with 10, -5 dies	5, 10

Return:

[5,10]

20.5 Collision Condition

Collision only happens when:

$st.back() > 0 \ \&\& \ asteroid < 0$

Examples with no collision:

[-2, 1]
[1, 2]
[-2, -1]

20.6 Complexity

Time:

$O(n)$

Each asteroid is pushed once and popped at most once.

Space:

$O(n)$

21. Full Problem Walkthrough 10 — LeetCode 232: Implement Queue using Stacks

21.1 Problem Summary

Implement queue operations using two stacks.

Queue is First In, First Out.

Stack is Last In, First Out.

Operations:

```
push(x)
pop()
peek()
empty()
```

21.2 Core Idea

Use two stacks:

- inStack for new elements
- outStack for removing elements

When outStack is empty, move everything from inStack to outStack.

This reverses order and gives queue behavior.

21.3 C++ Code

```
class MyQueue {
private:
    stack<int> inStack;
    stack<int> outStack;

    void moveIfNeeded() {
        if (outStack.empty()) {
            while (!inStack.empty()) {
                outStack.push(inStack.top());
                inStack.pop();
            }
        }
    }
}
```

```

public:
    MyQueue() {
    }

    void push(int x) {
        inStack.push(x);
    }

    int pop() {
        moveIfNeeded();

        int value = outStack.top();
        outStack.pop();

        return value;
    }

    int peek() {
        moveIfNeeded();
        return outStack.top();
    }

    bool empty() {
        return inStack.empty() && outStack.empty();
    }
};

```

21.4 Complexity

Each element moves from inStack to outStack at most once.

Amortized time per operation:

$O(1)$

Worst-case single pop or peek:

$O(n)$

Space:

$O(n)$

22. Full Problem Walkthrough 11 — LeetCode 225: Implement Stack using Queues

22.1 Problem Summary

Implement stack using queues.
Stack must behave Last In, First Out.

22.2 One Queue Approach

When pushing a new element:

21. Push it to queue.
22. Rotate previous elements behind it.

This makes the newest element appear at the front.

22.3 C++ Code

```
class MyStack {
private:
    queue<int> q;

public:
    MyStack() {
    }

    void push(int x) {
        q.push(x);

        int rotations = q.size() - 1;

        while (rotations-- > 0) {
            q.push(q.front());
            q.pop();
        }
    }

    int pop() {
        int value = q.front();
        q.pop();
        return value;
    }

    int top() {
```

```
        return q.front();
    }

    bool empty() {
        return q.empty();
    }
};
```

22.4 Complexity

push:

$O(n)$

pop, top, empty:

$O(1)$

Space:

$O(n)$

23. Full Problem Walkthrough 12 — LeetCode 227: Basic Calculator II

23.1 Problem Summary

Evaluate a string expression containing:

- non-negative integers
- +
- -
- *
- /
- spaces

Example:

`s = "3+2*2"`

Output:

7

23.2 How to Identify Stack

Multiplication and division have higher priority.

One simple stack solution:

- Push numbers for +
- Push negative numbers for -
- Immediately resolve * and / with stack top

23.3 C++ Code

```
class Solution {
public:
    int calculate(string s) {
        vector<int> st;
        long long number = 0;
        char operation = '+';

        for (int i = 0; i < s.size(); i++) {
            char c = s[i];

            if (isdigit(c)) {
                number = number * 10 + (c - '0');
            }

            if ((!isdigit(c) && c != ' ') || i == s.size() - 1) {
                if (operation == '+') {
                    st.push_back(number);
                } else if (operation == '-') {
                    st.push_back(-number);
                } else if (operation == '*') {
                    st.back() = st.back() * number;
                } else if (operation == '/') {
                    st.back() = st.back() / number;
                }

                operation = c;
                number = 0;
            }
        }

        int result = 0;

        for (int x : st) {
            result += x;
        }

        return result;
    }
};
```

23.4 Dry Run

Input:

"3+2*2"

3	push 3	3
+	operation becomes +	3
2	push 2	3, 2
*	operation becomes *	3, 2
2	multiply top: 2 * 2	3, 4

Sum:

7

23.5 Complexity

Time:

$O(n)$

Space:

$O(n)$

23.6 Common Mistakes

- Ignoring operator precedence
- Not processing final number
- Mishandling spaces
- Integer division order

24. Mini Problem 1 — LeetCode 1544: Make The String Great

Pattern

String as stack.

```
class Solution {
public:
    string makeGood(string s) {
        string st;

        for (char c : s) {
            if (!st.empty() && abs(st.back() - c) == 32) {
```

```
        st.pop_back();
    } else {
        st.push_back(c);
    }
}

return st;
}
};
```

Key idea:

Lowercase and uppercase versions of same English letter differ by 32 in ASCII.

25. Mini Problem 2 — LeetCode 1021: Remove Outermost Parentheses

Pattern

Track depth.

```
class Solution {
public:
    string removeOuterParentheses(string s) {
        string result;
        int depth = 0;

        for (char c : s) {
            if (c == '(') {
                if (depth > 0) {
                    result += c;
                }

                depth++;
            } else {
                depth--;

                if (depth > 0) {
                    result += c;
                }
            }
        }
    }
}
```

```
        return result;
    }
};
```

This uses stack-like depth, even without explicit stack.

26. Mini Problem 3 — LeetCode 921: Minimum Add to Make Parentheses Valid

Pattern

Count unmatched openings and needed closings.

```
class Solution {
public:
    int minAddToMakeValid(string s) {
        int open = 0;
        int additions = 0;

        for (char c : s) {
            if (c == '(') {
                open++;
            } else {
                if (open > 0) {
                    open--;
                } else {
                    additions++;
                }
            }
        }

        return additions + open;
    }
};
```

Stack can solve this, but counters are enough.

27. Mini Problem 4 — LeetCode 1249: Minimum Remove to Make Valid Parentheses

Pattern

Stack of indices.

```
class Solution {
public:
    string minRemoveToMakeValid(string s) {
        stack<int> st;
        vector<bool> remove(s.size(), false);

        for (int i = 0; i < s.size(); i++) {
            if (s[i] == '(') {
                st.push(i);
            } else if (s[i] == ')') {
                if (!st.empty()) {
                    st.pop();
                } else {
                    remove[i] = true;
                }
            }
        }

        while (!st.empty()) {
            remove[st.top()] = true;
            st.pop();
        }

        string result;

        for (int i = 0; i < s.size(); i++) {
            if (!remove[i]) {
                result += s[i];
            }
        }

        return result;
    }
};
```

28. Common Edge Cases for Stack Problems

28.1 Empty Stack

Always check before calling:

```
st.top();  
st.pop();
```

28.2 Empty Input

Stack problems often return:

- true for valid empty parentheses
- empty string for processed empty strings
- 0 for score/evaluation-style problems depending on statement

28.3 Leftover Items

For parentheses, leftover opening brackets mean invalid.

```
return st.empty();
```

28.4 Operator Order

For expression problems, subtraction and division require correct operand order.

28.5 Multi-Digit Numbers

Decode String and Calculator problems often have numbers like:

```
100[abc]
```

Do not assume one digit.

28.6 Nested Structures

Nested brackets require storing previous state, not just current state.

28.7 Duplicate Values

Min Stack must handle duplicate minimum values correctly.

29. Common Mistakes and Wrong Code

Mistake 1 — Calling top() on Empty Stack

Wrong:

```
if (st.top() == '(') {  
    st.pop();  
}
```

Correct:

```
if (!st.empty() && st.top() == '(') {  
    st.pop();  
}
```

Mistake 2 — Forgetting Final Stack Check

For Valid Parentheses:

```
return true;
```

is wrong.

Need:

```
return st.empty();
```

Mistake 3 — Using Queue Instead of Stack

Parentheses need most recent opening bracket.

Queue gives oldest opening bracket.

Wrong structure.

Mistake 4 — Reversing Operand Order

For RPN:

```
int b = st.top(); st.pop();
```

```
int a = st.top(); st.pop();
```

```
st.push(a - b);
```

not:

```
st.push(b - a);
```

Mistake 5 — Not Handling Multi-Digit Numbers

Wrong for Decode String:

```
number = c - '0';
```

Correct:

```
number = number * 10 + (c - '0');
```

Mistake 6 — Thinking Stack Is Sorted

Stack has no sorting property.

It only gives most recent item.

30. Pattern Recognition Checklist

Ask:

23. Do I need to match opening and closing symbols?
24. Does the problem involve nested structure?
25. Does the most recent item get resolved first?
26. Is there an undo/backspace/cancel operation?
27. Do I need to process previous values before current?
28. Does the problem involve expression evaluation?
29. Does the current item collide with the previous survivor?
30. Do I only care about the top unresolved item?
31. Would recursion solve this naturally?
32. Can an explicit stack simulate that recursion?

If yes, stack is likely useful.

31. Interview Explanation Templates

31.1 Valid Parentheses

I use a stack to store opening brackets. When I see a closing bracket, it must match the most recent opening bracket, which is at the top of the stack. If it does not match or the stack is empty, the string is invalid.

31.2 Min Stack

I maintain a normal stack for values and another stack for the minimum value at each level. This lets `getMin()` return the current minimum in $O(1)$.

31.3 Backspace Simulation

I treat the result as a stack. Normal characters are pushed, and a backspace pops the most recent character if one exists.

31.4 Decode String

When I enter a bracket, I save the current string and repeat count on stacks. When I close a bracket, I pop the previous state and append the repeated decoded substring.

31.5 Expression Evaluation

For postfix expressions, operands are pushed onto a stack. When an operator appears, I pop the most recent two operands, apply the operator, and push the result back.

32. Complexity Patterns

Valid Parentheses	$O(n)$	$O(n)$
Min Stack	$O(1)$ per operation	$O(n)$
Baseball Game	$O(n)$	$O(n)$
Backspace String Compare	$O(n + m)$	$O(n + m)$
Remove Adjacent Duplicates	$O(n)$	$O(n)$
Simplify Path	$O(n)$	$O(n)$
Evaluate RPN	$O(n)$	$O(n)$
Decode String	$O(\text{output length})$	$O(\text{output length})$
Asteroid Collision	$O(n)$	$O(n)$
Queue using Stacks	amortized $O(1)$	$O(n)$
Basic Calculator II	$O(n)$	$O(n)$

33. Time-Boxed Practice Guidance

Easy Stack Problems

Target:

10–20 minutes

Examples:

- Valid Parentheses
- Baseball Game
- Backspace String Compare
- Remove Adjacent Duplicates
- Make The String Great

Medium Stack Problems

Target:

20–40 minutes

Examples:

- Min Stack
- Simplify Path
- Evaluate Reverse Polish Notation
- Asteroid Collision
- Decode String

Harder Stack / Parsing Problems

Target:

40–60 minutes

Examples:

- Basic Calculator
 - Basic Calculator II
 - Number of Atoms
 - Remove Invalid Parentheses
 - Mini Parser
-

34. Expanded Practice Set

34.1 Must-Do Easy

33. Valid Parentheses
34. Baseball Game
35. Backspace String Compare
36. Remove All Adjacent Duplicates in String
37. Make The String Great
38. Remove Outermost Parentheses
39. Minimum Add to Make Parentheses Valid

34.2 Must-Do Medium

40. Min Stack
41. Simplify Path
42. Evaluate Reverse Polish Notation

- 43. Decode String
- 44. Asteroid Collision
- 45. Implement Queue using Stacks
- 46. Implement Stack using Queues
- 47. Minimum Remove to Make Valid Parentheses
- 48. Basic Calculator II

34.3 Strong Interview Practice

- 49. Basic Calculator
- 50. Basic Calculator III
- 51. Remove All Adjacent Duplicates in String II
- 52. Score of Parentheses
- 53. Online Stock Span, monotonic stack
- 54. Daily Temperatures, monotonic stack
- 55. Exclusive Time of Functions
- 56. Validate Stack Sequences

34.4 Hard / Stretch

- 57. Number of Atoms
 - 58. Remove Invalid Parentheses
 - 59. Longest Valid Parentheses
 - 60. Mini Parser
 - 61. Parsing Boolean Expression
 - 62. Basic Calculator IV
-

35. Suggested Study Order

Step 1 — Basic Stack Behavior

Start with:

- 63. Valid Parentheses
- 64. Baseball Game
- 65. Backspace String Compare

Goal:

Understand push, pop, top.

Step 2 — String as Stack

Then solve:

- 66. Remove Adjacent Duplicates
- 67. Make The String Great
- 68. Minimum Remove to Make Valid Parentheses

Goal:

Learn how stack builds valid strings.

Step 3 — Stack Design

Solve:

- 69. Min Stack
- 70. Queue using Stacks
- 71. Stack using Queues

Goal:

Understand stack-based data structure design.

Step 4 — Parsing and Paths

Solve:

- 72. Simplify Path
- 73. Decode String
- 74. Evaluate Reverse Polish Notation

Goal:

Learn stack for parsing and nested structure.

Step 5 — Harder Simulation

Solve:

- 75. Asteroid Collision
- 76. Basic Calculator II
- 77. Longest Valid Parentheses

Goal:

Prepare for advanced stack and parsing problems.

36. Pattern Decision Table

opening/closing brackets	matching stack	Valid Parentheses
backspace/undo	string stack	Backspace String
remove adjacent duplicates	string stack	LC 1047
get minimum in stack	auxiliary min stack	Min Stack
Unix path	directory stack	Simplify Path
postfix expression	operand stack	Evaluate RPN
nested encoded string	count/string stacks	Decode String
collision with previous survivor	survivor stack	Asteroid Collision
queue from stacks	two stacks	Implement Queue
calculator with precedence	number stack	Basic Calculator II
unmatched parentheses index	index stack	Minimum Remove

37. Final Cheat Sheet

Basic Stack

```
stack<int> st;
```

```
st.push(x);
st.top();
st.pop();
st.empty();
```

String Stack

```
string st;

for (char c : s) {
    if (!st.empty() && shouldRemove(st.back(), c)) {
        st.pop_back();
    } else {
        st.push_back(c);
    }
}
```

Valid Parentheses

```
for (char c : s) {
    if (c == '(' || c == '[' || c == '{') {
        st.push(c);
    } else {
        if (st.empty()) {
            return false;
        }

        char top = st.top();
        st.pop();

        if (!matches(top, c)) {
            return false;
        }
    }
}

return st.empty();
```

Min Stack

```
st.push(val);
minSt.push(min(val, minSt.empty() ? val : minSt.top()));
```

RPN

```
int b = st.top(); st.pop();
int a = st.top(); st.pop();
```

```
st.push(apply(a, b, op));
```

Decode String

```
countStack.push(number);  
stringStack.push(current);
```

On]:

```
repeat current string  
append to previous string
```

Asteroid Collision Condition

```
while (!st.empty() && st.back() > 0 && asteroid < 0) {  
    // collide  
}
```

38. What to Master Before Chapter 14

Before moving to Monotonic Stack, make sure you can:

- Use C++ stack
- Use vector as a stack
- Use string as a stack
- Check empty stack before top()
- Solve Valid Parentheses
- Implement Min Stack
- Simulate backspace with stack
- Simplify Unix paths
- Evaluate Reverse Polish Notation
- Decode nested strings
- Solve Asteroid Collision
- Know when stack is better than queue
- Explain Last In, First Out clearly
- Recognize unresolved-recent-item problems

39. Core Takeaway

A stack is useful when the most recent unresolved item must be processed first.
The core idea is:

```
push when something starts  
pop when the most recent thing is resolved
```

Stacks appear in:

- parentheses
- undo operations

- expression evaluation
- parsing
- nested structures
- path simplification
- collision simulation

The key interview question is:

Does the current item interact with the most recent previous unresolved item?

If yes, stack is probably the right pattern.

Next chapter builds on this idea with **Monotonic Stack**, where the stack is not just LIFO, but also kept in increasing or decreasing order to solve next greater / next smaller problems efficiently.
